



CITY UNIVERSITY MALAYSIA

OBJECT-ORIENTED PROGRAMMING FUNDAMENTALS IN JAVA

ASSIGNMENT 1 - INDIVIDUAL SELF-REFLECTIVE REPORT

STUDENT NAME	Radif Hasan
STUDENT ID	202409010255
COURSE	BIT1123 Object-Oriented Programming
CLASS CODE	BIT1123
PROGRAMME	Bachelor of Computer Science (Software Engineering) (Hons) Major in Cyber Security
CAMPUS	Cyberjaya Campus

Repository: github.com/radifmalaysia2-spec/ASSIGNMENT1_OOP_202409010255

Contents

1	Introduction
2	Learning Journey: Week 1 to Week 10
3	Challenges Encountered
4	How I Overcame the Challenges
5	Improvement in Java Programming Skills
6	Understanding of Object-Oriented Programming
7	Future Learning Plans
8	Conclusion
9	GitHub Repository URL

This report reflects on the completed tutorial work and the practical evidence stored in the accompanying GitHub repository.

1. Introduction

This report reflects on my learning journey through the BIT1123 Object-Oriented Programming tutorials. At the beginning, I understood Java mainly as a language for writing instructions in sequence. The weekly exercises gradually changed that view. I learned to model a problem using objects, decide which data belongs inside a class, control access to that data, and reuse behaviour through inheritance and abstraction.

The tutorials also required more than writing code. I had to keep the work in a consistent GitHub repository, use meaningful file and folder names, write instructions that another person could follow, and verify that each command-line program produced the intended result. These tasks helped me connect classroom concepts with the way software projects are organised in practice.

2. Learning Journey: Week 1 to Week 10

Week 1 - Java Structure, Input and GitHub

I began by setting up the repository and practising the basic Git workflow. The two programs introduced the structure of a Java class and the main method. HelloWorld showed the simplest compile-and-run cycle. StudentGrade added keyboard input, numeric validation, if-else selection, and formatted output. This week made the relationship between source code, compilation, and execution much clearer to me.

Week 2 - Classes and Objects

The Student exercise was my first complete model of an object with attributes and behaviour. I used a constructor to initialise the student's name, age, and GPA, then called methods through a Student object. I understood that a class is a design, while an object is a specific instance created from that design. Separating Student from Main also showed why a program is easier to understand when responsibilities are divided across files.

Weeks 3-4 - Inheritance and Polymorphism

The university people scenario introduced a parent Person class and the Student and Lecturer subclasses. Common data such as name and ID stayed in Person, while each subclass supplied its own version of introduce(). The strongest lesson was that a Person reference could point to different subclasses and still call the correct overridden method. This gave me a practical understanding of runtime polymorphism instead of only memorising its definition.

Week 5 - Encapsulation

The student information system required private fields and public getters and setters. I learned that encapsulation is not simply hiding variables; it creates a controlled boundary around an object's state. The CGPA setter validates the value before accepting it, which prevents an invalid state. Writing the accompanying answers also helped me explain why public fields make a program more difficult to protect and maintain.

2. Learning Journey: Week 1 to Week 10 (continued)

Week 6 - Reusing and Extending a Class

The Lecturer class reused the ID and name from Employee and added a subject and department. The super call showed how a subclass passes shared information to its parent constructor. I also saw that inheritance should express a real relationship: a lecturer is an employee, so the model is logical as well as technically correct.

Week 7 - Abstraction

The appliance system used an abstract class for behaviour shared by every appliance. Brand display, power on, and power off were implemented once, while operate() remained abstract because every appliance works differently. I completed washing machine and refrigerator examples and added an air conditioner and television. Processing the objects through one Appliances array demonstrated how abstraction and polymorphism work together.

Weeks 8-9 - Collections and File Handling

The to-do application used an ArrayList to keep three tasks and buffered streams to save and reload them. I learned that memory and permanent storage are different: the list exists while the program runs, but the text file keeps the information afterwards. Try-with-resources was useful because it closes readers and writers safely, while IOException handling gives the user a meaningful message if a file operation fails.

Week 10 - Swing and Event Handling

The final exercise moved from terminal output to a graphical quiz. Questions stores the question, two options, and correct answer, while QuizBattleGUI builds the window and handles button clicks. The ActionListener connects a user's action to program logic. This tutorial brought several ideas together: objects, encapsulated data, composition, methods, conditions, and event-driven execution.

3. Challenges Encountered

Challenge	What made it difficult
Matching constructors and objects	When a constructor accepted several values, the order and data types had to match the object creation statement exactly. A mismatch caused compilation errors or stored the wrong value.
Understanding overriding	At first, inheritance and overriding seemed similar to copying methods. The Person example showed that the subclass replaces behaviour while keeping the same method signature.
Protecting object state	Private fields could not be accessed directly from Main. This initially felt restrictive, but it exposed why controlled methods are necessary and why validation belongs inside the class.
Reading and writing files	File operations introduced checked exceptions and resources that must be closed. The program also needed a clear order: collect tasks, write the file, reopen it, and display its contents.
Building a GUI	Swing required precise component creation, placement, event registration, and state updates. Unlike a terminal program, the code waits for user events instead of finishing from top to bottom.

4. How I Overcame the Challenges

I dealt with compilation problems by reading the first error carefully, checking the filename against the public class name, and comparing method parameters with the values passed from Main. Fixing the earliest error was important because one small problem could produce several later messages.

For inheritance and overriding, I traced the program from the declared reference type to the actual object type. I also kept the `@Override` annotation because the compiler can then warn me if the method signature is wrong. For encapsulation, I moved every change to student data through a setter and added a valid range check for CGPA.

For file handling, I divided the program into small `saveTasks()` and `displaySavedTasks()` methods. Try-with-resources reduced the chance of leaving a file open. In the Swing exercise, I separated the quiz data from the window and used one event method for both buttons. This made the user-interface code easier to follow and prevented repeated logic.

5. Improvement in Java Programming Skills

My main improvement is the ability to design a solution before writing the main method. I now identify the important entities, choose their fields, define constructors, and decide which behaviour belongs to each class. I am more comfortable using access modifiers, arrays and ArrayLists, loops, method calls, exceptions, and standard library classes.

Area	Earlier approach	Current approach
Program structure	Most logic placed in main	Responsibilities divided into classes and methods
Data access	Fields accessed directly	Private state accessed through controlled methods
Code reuse	Similar code repeated	Shared behaviour placed in a parent or abstract class
Problem solving	Started coding immediately	Model the objects and relationships first
Reliability	Assumed input and files would work	Validate values and handle file errors
Documentation	Code stored without context	Each folder explains purpose and run commands

6. Understanding of Object-Oriented Programming

I now understand OOP as a way to organise software around responsibilities and relationships. Encapsulation protects an object's valid state. Inheritance reuses genuinely shared features. Polymorphism lets the same operation produce behaviour appropriate to the actual object. Abstraction exposes the operations users need while leaving different classes free to implement their internal work differently.

The tutorials also showed that these principles are strongest when used together. In the appliance program, the abstract parent defines a common interface, subclasses implement different operations, and an array processes every object through the same type. In the quiz, the question data is separated from the graphical interface, so the model and presentation have distinct responsibilities.

7. Future Learning Plans

1	Practise interfaces, composition, packages, and generic collections through small projects.
2	Add unit tests with JUnit so that class behaviour can be checked automatically.
3	Learn JavaFX and improve interface layout beyond absolute Swing positioning.
4	Connect Java applications to a database using JDBC and apply exception handling carefully.
5	Study design principles such as single responsibility and favouring composition when inheritance is not a natural relationship.
6	Continue using Git branches and clear commit messages for future coursework and portfolio projects.

8. Conclusion

The tutorial sequence gave me a practical foundation in Java and object-oriented programming. I progressed from writing simple statements to designing related classes, protecting data, sharing behaviour, saving information, and responding to events in a graphical application. The most valuable outcome is that I can now explain why a design is organised in a certain way instead of only making the program produce output.

Organising every tutorial in one GitHub repository also made the progress visible. The folders, source files, supporting handouts, run instructions, and reflective report form a record that I can revisit and improve as I learn more advanced Java development.

9. GitHub Repository URL

https://github.com/radifmalaysia2-spec/ASSIGNMENT1_OOP_202409010255

Declaration: This reflection is based on the tutorial programs and supporting materials included in the repository.